

Gym Tracker App

Product Specifications & Architecture Blueprint

1. Executive Summary

This document outlines the functional specifications and structural logic for a mobile gym tracking application (Android native, with future macOS compatibility via Flutter/Dart). The core value proposition is to provide users with an offline-first, highly responsive tool to manage custom workout routines, log live session data, and track long-term performance progression.

2. Core Feature Specifications

A. Exercise Library & Management

The application relies on a robust, locally stored SQLite database initialized with 150+ pre-mapped exercises. The user interaction layer must support:

- **Search & Filter Engine:** Users must be able to query the database using text input (fuzzy search on exercise name) or categorical filters. The relational database schema allows filtering by primary muscle group (e.g., "Chest") or implement type (e.g., "Barbell", "Cable").
- **Custom Exercise Injection:** Recognizing that the seeded database cannot cover every niche movement, the app provides a "Create Exercise" flow. Users can input a custom name, select an implement, and map primary/secondary muscle involvements. This data

is written directly to the local SQLite `Exercises` and `Exercise\_Muscles` tables, making the custom movement instantly searchable and usable across all other features.

B. The Dual Workout Engine

The application supports two distinct training modalities, feeding into the same active session architecture:

- **Mode 1: Ad-Hoc (Free Play):** Designed for dynamic gym environments. The user initiates a blank session and utilizes the Search & Filter Engine to add exercises sequentially as they navigate the gym floor.
- **Mode 2: Multi-Day Program Templates:** Designed for structured training (e.g., Push/Pull/Legs). The database utilizes `Programs`, `Program\_Days`, and `Program\_Day\_Exercises` tables. Users select a specific day within a template (e.g., "PPL - Day 1: Push"). The app queries the relational tables to automatically populate the Active Workout Session screen with the predefined exercises, order, and target set/rep ranges.

C. The Active Session Interface (Core UX)

This is the primary user interface where data logging occurs. It is accessed via both Ad-Hoc and Template modes.

- **Performance Logging:** Each exercise in the session presents a dynamic list of input fields for logging Sets. The required inputs are "Weight" (numerical, allowing decimals) and "Reps" (integer). Upon confirming a set, the data is committed to the `Sets` table linked to the current `Session\_ID`.
- **Decoupled Manual Rest Timer:** A critical UX decision. The timer does *not* trigger automatically upon logging a set. Instead, it is an independent, persistent UI element (floating or pinned) accessible throughout the active session. It features manual Play, Pause, and Reset controls, granting the user absolute agency over their rest periods regardless of when data entry occurs.

UX Note: *Decoupling the timer prevents user frustration associated with delayed data entry or interruptions during training, ensuring the app remains a passive tool rather than an active taskmaster.*

D. Analytics & History

The application leverages the relational SQL structure to generate meaningful feedback.

- **Historical Log View:** A chronological list or calendar interface querying the `Sessions` table. Users can review past dates to see completed exercises, logged metrics, and session notes.
- **Progression Visualization:** The app aggregates historical data from the `Sets` table grouped by `Exercise\_ID` and chronological `Date\_Time`. This data feeds into visual charts (e.g., line graphs) to map performance metrics such as 1-Rep Max progression or Total Volume lifted over a selected timeframe.

3. App Flow Architecture

The following diagram represents the user navigation logic. Note how the dual workout engines converge to utilize a single, unified Active Session UI, ensuring a DRY (Don't Repeat Yourself) codebase architecture.

Dashboard (Home)



Start Ad-Hoc Workout

Select Multi-Day Program
(e.g., PPL -> Day 1)



Active Workout Session

- Log Sets (Weight/Reps)
- Operate Manual Rest Timer



Save Session Data to SQLite DB



Historical Logs & Analytics Dashboard

Secondary Navigation Paths:

Dashboard → Exercise Library



Search / Filter DB

Add Custom Exercise to DB